

number for the parent directory. Thus, a procedure looking up `./chris/prog.c` simply looks up `..` in the working directory, finds the i-node number for the parent directory, and searches that directory for `chris`. No special mechanism is needed to handle these names. As far as the directory system is concerned, they are just ordinary ASCII strings, just the same as any other names. The only bit of trickery here is that `..` in the root directory points to itself.

4.6 RESEARCH ON FILE SYSTEMS

File systems have always attracted more research than other parts of the operating system and that is still the case. Entire conferences such as FAST, MSST, and NAS are devoted largely to file and storage systems.

A considerable amount of research addresses the reliability of storage and file systems. A powerful way to guarantee reliability is to formally *prove* the safety of your system even in the face of catastrophic events, such as crashes (Chen et al., 2017). Also, with the rising popularity of SSDs as the primary storage medium, it is interesting to look at how well they hold up in large enterprise storage systems (Maneas et al., 2020).

As we have seen in this chapter, file systems are complex beasts and developing new file systems is not easy. Many operating systems allow file systems to be developed in user space (e.g., the FUSE userspace filesystem framework on Linux), but the performance is generally much lower. With new storage devices such as low-level SSDs arriving on the market, the need for an agile storage stack is important and research is needed to develop high-performance file systems quickly (Miller et al., 2021). In fact, new advances in storage technology is driving much of the research on file systems. For instance, how do we build efficient file systems for new persistent memory (Chen et al., 2021; and Neal, 2021)? Or how can we speed up file system checking (Domingo, 2021)? Even fragmentation creates different issues on hard disks and SSDs and requires different approaches (Kesavan, 2019).

Storing increasing amounts of data on the same file system is challenging, especially in mobile devices, leading to the development of new methods to compress the data without slowing down the system too much, for instance by taking the access patterns of files into account (Ji et al., 2021). We have seen that as an alternative to per-file or per-block compression, some file systems today support deduplication across the entire system to prevent storing the same data twice. Unfortunately, deduplication tends to lead to poor data locality and trying to obtain good deduplication without performance loss due to lack of locality is difficult (Zou, 2021). Of course, with data deduplicated all over the place, it becomes much harder to estimate how much space is left or will be left when we delete a certain file (Harnik, 2019).

4.7 SUMMARY

When seen from the outside, a file system is a collection of files and directories, plus operations on them. Files can be read and written, directories can be created and destroyed, and files can be moved from directory to directory. Most modern file systems support a hierarchical directory system in which directories may have subdirectories and these may have subsubdirectories ad infinitum.

When seen from the inside, a file system looks quite different. The file system designers have to be concerned with how storage is allocated and how the system keeps track of which block goes with which file. Possibilities include contiguous files, linked lists, file-allocation tables, and i-nodes. Different systems have different directory structures. Attributes can go in the directories or somewhere else (e.g., an i-node). Disk space can be managed using free lists or bitmaps. File-system reliability is enhanced by making incremental dumps and by having a program that can repair sick file systems. File-system performance is important and can be enhanced in several ways, including caching, read ahead, and carefully placing the blocks of a file close to each other. Log-structured file systems also improve performance by doing writes in large units.

Examples of file systems include ISO 9660, MS-DOS, and UNIX. These differ in many ways, including how they keep track of which blocks go with which file, directory structure, and management of free disk space.

PROBLEMS

1. In Windows, when a user double clicks on a file listed by Windows Explorer, a program is run and given that file as a parameter. List two different ways the operating system could know which program to run.
2. In early UNIX systems, executable files (*a.out* files) began with a very specific magic number, not one chosen at random. These files began with a header, followed by the text and data segments. Why do you think a very specific number was chosen for executable files, whereas other file types had a more-or-less random magic number as the first word?
3. In Fig. 4-5, one of the attributes is the record length. Why does the operating system ever care about this?
4. Is the `open` system call in UNIX absolutely essential? What would the consequences be of not having it?
5. Systems that support sequential files always have an operation to rewind files. Do systems that support random-access files need this, too?
6. Some operating systems provide a system call `rename` to give a file a new name. Is there any difference at all between using this call to rename a file and just copying the file to a new file with the new name, followed by deleting the old one?